

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Real-Time AI Chat Assistant with LangChain & PyTorch

Detailed project brief generated from the project record.

CANDIDATE

Om Ravindra Bhondve

DURATION

7 days

TECHNOLOGY STACK

Bash

Git

Postman

Python

REST API

WebSockets

LangGraph

LangChain

GitLab CI

GitHub

GitHub Actions

PyTorch

MongoDB

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

Build a lightweight microservice that answers user questions using a small PyTorch transformer model wrapped by LangChain. The service exposes a REST endpoint and a WebSocket for real time chat, logs conversations in MongoDB, and can be invoked via Postman. The project showcases data flow, model inference, API design, and basic CI.

CANDIDATE Om Ravindra Bhondve	TYPE ai_ml	DURATION 7 days
---	----------------------------------	---------------------------------------

Primary Objectives

- Implement a Python service that loads a pretrained PyTorch model and generates answers via LangChain.
- Expose a REST endpoint (/chat) and a WebSocket (/ws) for synchronous and real time interactions.
- Persist each conversation turn in MongoDB for later analysis.
- Automate build, test, and containerization with a simple CI workflow.

Candidate Details

NAME

Om Ravindra Bhondve

Technology Stack

Technologies, frameworks and tools expected for implementation.

Bash Technology / Tool	Git Technology / Tool	Postman Technology / Tool
--	---	---

Python

Technology / Tool

REST API

Technology / Tool

WebSockets

Technology / Tool

LangGraph

Technology / Tool

LangChain

Technology / Tool

GitLab CI

Technology / Tool

GitHub

Technology / Tool

GitHub Actions

Technology / Tool

PyTorch

Technology / Tool

MongoDB

Technology / Tool

Functional Requirements

- POST /chat with {"message":"..."} returns a generated answer.
- WebSocket connection receives user messages and streams model responses.
- Conversation logs store user_id, timestamp, message, and answer in MongoDB.
- API returns HTTP 400 for empty or malformed requests.
- A Bash script (run.sh) starts the service and initializes the DB collection.
- CI pipeline runs unit tests and builds a Docker image.

Non-Functional Requirements

- Model inference latency must be under 2 seconds for a single request.
- API must handle at least 5 concurrent WebSocket connections.
- All secrets (e.g., DB URI) are read from environment variables.
- Code style follows PEP8 and is linted in CI.
- Service must log errors to stdout for container visibility.

Modules & Features

Major functional areas that make up the project.

Model Service

Loads the PyTorch model and provides a LangChain wrapper for inference.

- Load pretrained transformer from torch.hub
- Wrap model with LangChain LLM interface
- Expose generate_answer(text) function

API Layer

FastAPI server offering REST and WebSocket endpoints.

- POST /chat endpoint
- WebSocket /ws endpoint with streaming
- Input validation with Pydantic

Persistence

MongoDB integration for conversation logging.

- Create collection "conversations"
- Insert log document per turn
- Index on user_id and timestamp

CI/CD

GitHub Actions workflow for testing and Docker image build.

- Run pytest suite
- Lint with flake8
- Build and push Docker image to GitHub Packages

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Setup & Foundations

Initialize Git repo and push to GitHub, Create Dockerfile and run.sh script, Add GitHub Actions workflow for lint and test, Document setup steps in README

- Initialize Git repo and push to GitHub
- Create Dockerfile and run.sh script
- Add GitHub Actions workflow for lint and test
- Document setup steps in README

2

Core Service Development

Load small pretrained PyTorch model (e.g., distilbert-base-uncased), Create LangChain LLM wrapper, Build FastAPI routes for REST and WebSocket, Integrate PyMongo client and define log schema, Write unit tests for generate_answer and API validation

- Load small pretrained PyTorch model (e.g., distilbert-base-uncased)
- Create LangChain LLM wrapper
- Build FastAPI routes for REST and WebSocket
- Integrate PyMongo client and define log schema
- Write unit tests for generate_answer and API validation

3

Testing, Documentation & Delivery

Run integration tests using TestClient, Create Postman collection for /chat and WebSocket, Update README with usage examples, Build and push Docker image via CI, Add Bash script to launch container locally

- Run integration tests using TestClient
- Create Postman collection for /chat and WebSocket
- Update README with usage examples
- Build and push Docker image via CI
- Add Bash script to launch container locally

API Design

Key API endpoints and expected responsibilities.

POST

/chat

Receive a user message and return a generated answer.

GET

/ws

Upgrade to WebSocket for real time bidirectional chat.

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correctness of Model Integration | Description: Model loads without errors and generates sensible answers. | Weight: 20
- Criterion: API Functionality | Description: REST and WebSocket endpoints behave as specified and handle errors. | Weight: 20
- Criterion: Code Quality & CI | Description: Code follows PEP8, passes lint, and CI pipeline succeeds. | Weight: 20

- Criterion: Documentation & Usability | Description: README, setup scripts, and Postman collection enable easy execution. | Weight: 15
- Criterion: Testing Coverage | Description: Unit and integration tests cover core functions and edge cases. | Weight: 15
- Criterion: Security & Config Management | Description: Secrets are externalized, input is validated, and errors are not leaked. | Weight: 10

Expected Deliverables

Artifacts and outcomes expected at project completion.

- GitHub repository URL
- Docker image (public or GitHub Packages)
- README with setup and usage instructions
- Postman collection JSON
- GitHub Actions workflow file
- Test suite (pytest) with coverage report

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

Om already knows Python, Bash, Git, and database basics (MySQL/PostgreSQL). This project leverages those skills while introducing PyTorch, LangChain, FastAPI, and MongoDB—new but manageable technologies for a junior ML Engineer. The 7 day scope focuses on core model inference, API design, and CI, providing a realistic assessment of problem solving, system design, and implementation ability without overwhelming complexity.

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.